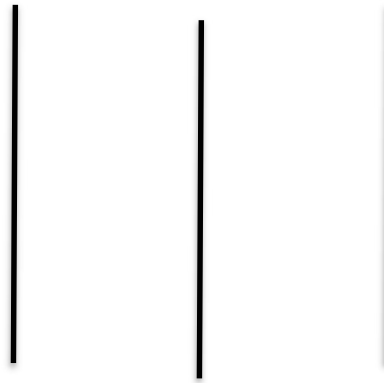




Universitat de Girona



IFROS 2024-2026
Autonomous System
Lab-5: q Learning



Delivered by:

Pravin Oli (u1999699)
Gebrecherkos Gebreslassie (u1999542)

Supervisor:

Narcís Palomeras Rovira
Marta Real Vial

Date of Submission:

25/12/2024

Table of Contents

1. Introduction	3
1.1. Reinforcement Learning	3
1.2. Q learning Algorithm Concept	3
2. Implementation	4
Code Organization in ipynb notebook	4
3. Output & Result	5
3.1 For given Map	5
3.2 For Created Map1	5
3.2 For Created Map2	6
4.Discussion	7
5.Problems Encountered	7
6. Conclusion	8

1. Introduction

1.1. Reinforcement Learning

It is defined as "Learning a mapping from situations to actions to maximize long-term reward, without using a model of the world".

The key elements of RL are:

Agent: The entity that takes actions.

Environment: The world the agent interacts with.

State: The current situation of the environment.

Action: The decision or choice the agent makes.

Reward: Feedback from the environment based on the action taken.

Policy: A strategy that maps states to actions.

Value Function: Estimates the long-term return of states or state-action pairs.

Types:

Model-based RL: The agent builds a model of the environment.

Model-free RL: The agent learns without explicitly modelling the environment.

Example Algorithms:

Q-learning

SARSA

Deep Q-Networks (DQN)

1.2. Q learning Algorithm Concept

Q-learning is a specific model-free RL algorithm that learns the value of taking a certain action in a particular state (state-action pair) using a Q-value function. Q-Value Function, $Q(s, a)$ estimates the cumulative future reward of taking action (a) in state (s) and following the optimal policy thereafter.

$$Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

Alpha (α) = Learning rate

Reward (r) = Immediate reward

Gamma (γ) = Discounted factor for future reward

$\max_{a'} Q(s', a')$ = estimate of optimal future reward

Initialize $Q(s, a)$ to 0 or random

For n episodes:

Initialize s randomly in any free cell

repeat:

Choose action 'a' following epsilon-greedy policy

Take action a, observe r and s'

$Q(s, a) \leftarrow Q(s, a) + \alpha * (r + \gamma * \max_{a'} Q(s', a')) - Q(s, a)$

$s \leftarrow s'$

until the episode terminates

endFor

2. Implementation

This lab is implemented in two parts. The notebook ipynb file is organized in similar way dividing the parts:

1. Part 1 Q learning implementation on given map
2. Part 2 New Maps with user selection

Code Organization in ipynb notebook

Part 1

Import Libraries:
Map Configuration:
(Plot map)

Environment Class:
init, reset, state, get_state,
render

Load and Display:
(Plot selected map with start & goal)

Q learning Class:
init, epsilon_greedy_policy,
episode, train, get _optimal_policy,
value_function

Training and Testing:
Parameters, train agent, plot average
reward

Value function and policy:
Plot value function and policy

Test current policy:
Generate animation

Part 2

User input map:
Select map 1 or 2

User input goal:
Set goal coordinates

User input parameters:
Set different parameters values.

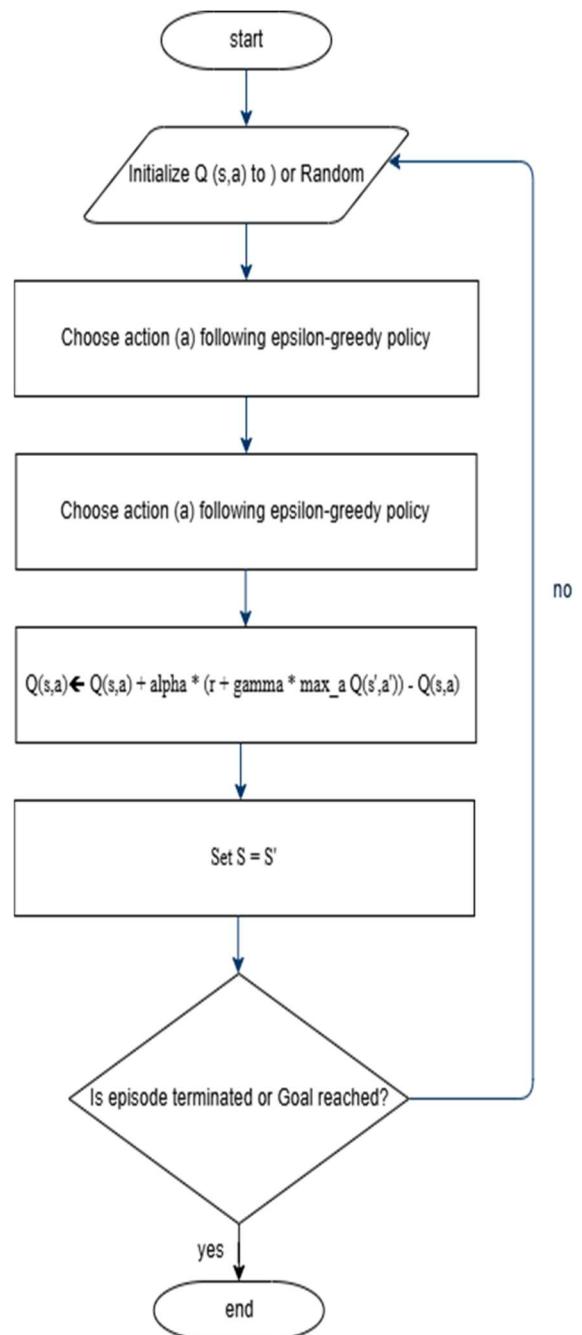
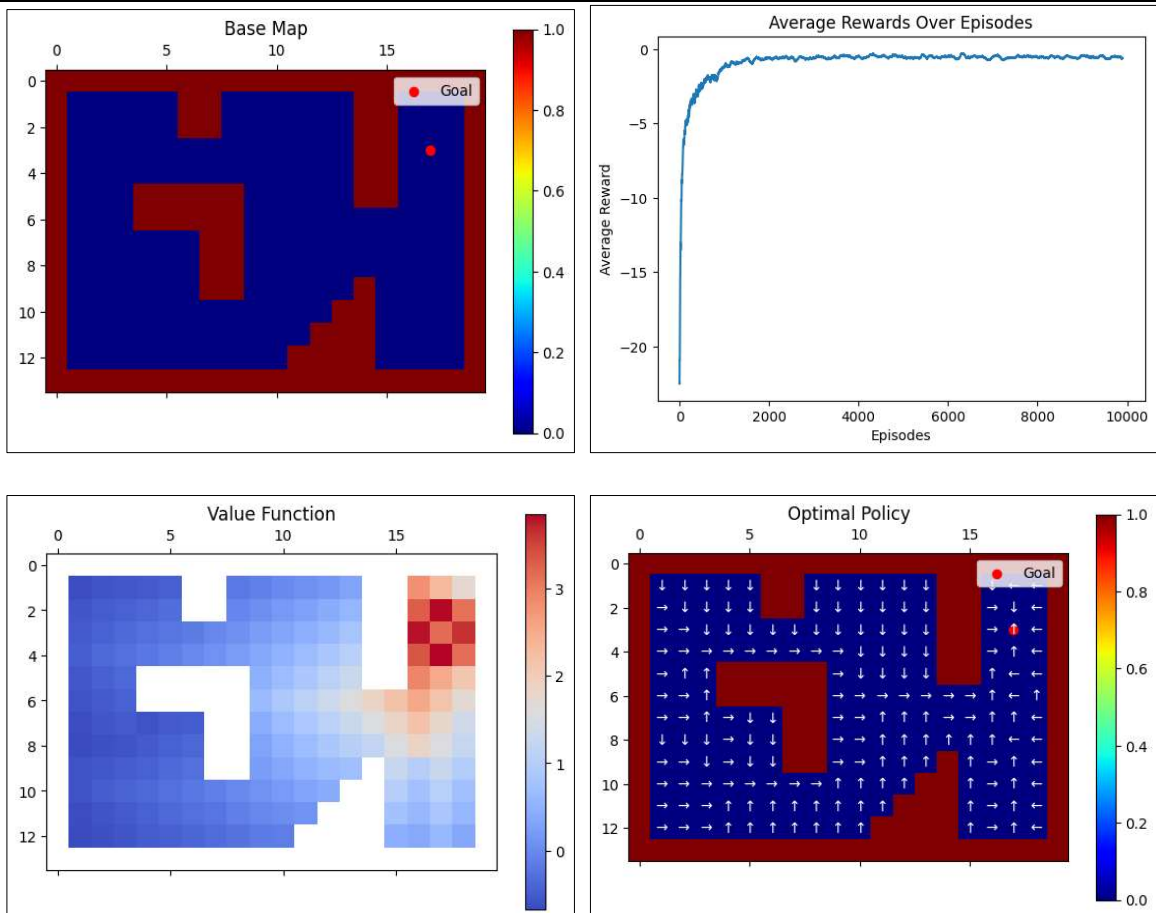


Figure 1 Flow chart for Q learning Algorithm

3. Output & Result

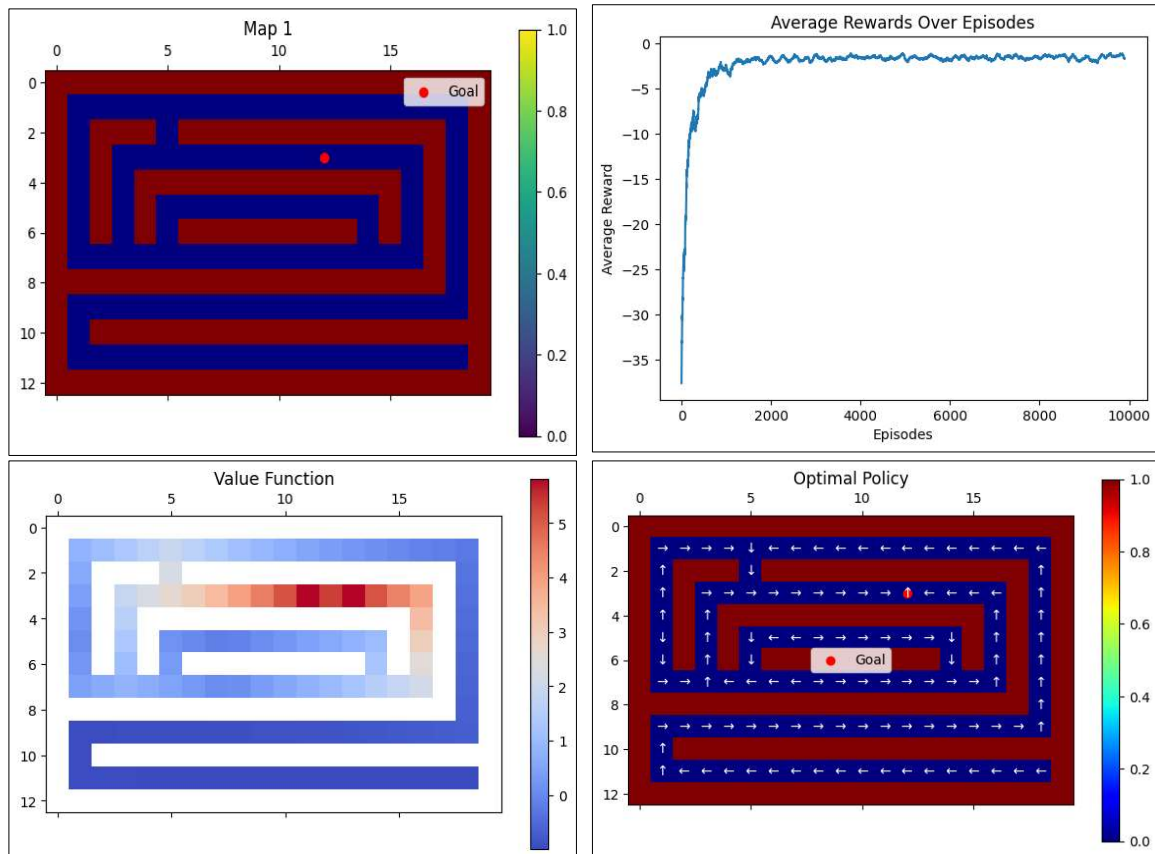
3.1 For given Map

```
Goal= (3,17)
Parameters:
# Training and testing parameters
alpha = 0.1
gamma = 0.9
epsilon = 0.1
n_episodes = 10000
max_steps = 1000
Episode 100/10000, Average Reward: -26.89
..
Episode 10000/10000, Average Reward: -0.51
```



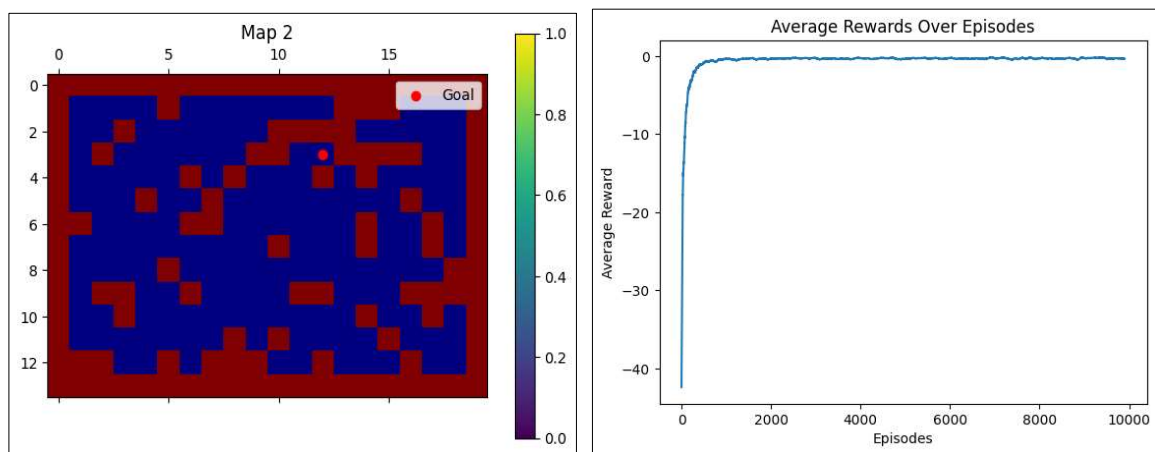
3.2 For Created Map1

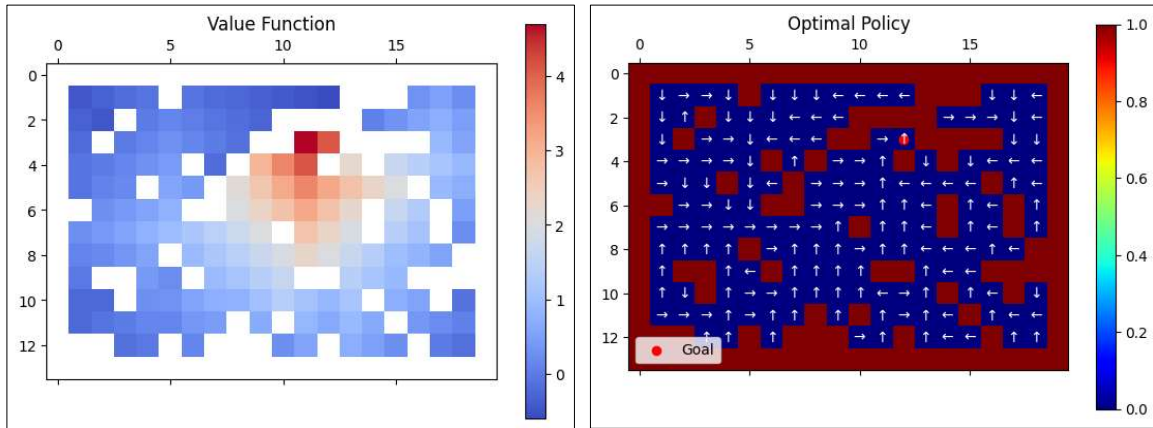
```
Goal= (3,12)
Parameters:
# Training and testing parameters
alpha = 0.1
gamma = 0.9
epsilon = 0.1
n_episodes = 10000
max_steps = 1000
Episode 100/10000, Average Reward: -37.62
..
Episode 10000/10000, Average Reward: -1.69
```



3.2 For Created Map2

Goal= (3,12)
Parameters:
Training and testing parameters
alpha = 0.1
gamma = 0.9
epsilon = 0.1
n_episodes = 10000
max_steps = 1000
Episode 100/10000, Average Reward: -42.37
..
Episode 10000/10000, Average Reward: -0.35





4. Discussion

Effect of Parameters:

S. N	Parameter	Low Value	High Value
1	α (Learning Rate)	Slow, stable learning; delayed convergence.	Fast learning; potential instability.
2	γ (Discount Factor)	Focuses on immediate rewards; suboptimal strategies.	Long-term focus; slower but more strategic learning.
3	ϵ (Epsilon)	Limited exploration; risk of local optima.	Thorough exploration; slower convergence.
4	n_episodes	Insufficient training; suboptimal policies.	Comprehensive training; better policies but diminishing returns.
5	max_steps	Insufficient time to solve episodes.	Better exploration and goal-reaching, but computationally expensive.

Hence, we should choose a balanced value for all parameters.
In this lab I have chosen:

alpha = 0.1
gamma = 0.9
epsilon = 0.1
n_episodes = 10000
max_steps = 1000

check_every_n=100 `check\_every\_n` episodes to print the average reward and store it.

5. Problems Encountered

Problem 1: Training and Testing Parameters

- Challenge: Choosing optimal values for alpha, gamma, epsilon, and balancing exploration vs. exploitation.

Solution: Used balanced values (alpha=0.1, gamma=0.9, epsilon=0.1, n_episodes = 10000 max_steps = 1000 check_every_n=100 check_every_n episodes to print the average reward and store it.)

Problem 2: Generating Animation Video

- Challenge: Long processing times and cluttered visuals for large maps or high max_steps.
- Solution: Optimized path tracking and efficient rendering with ani.to_jshtml() for animation.

Problem 3: Implementing on Different Maps with User Input

- Challenge: Ensuring valid user-selected goal positions and dynamic map support.
- Solution: Added validation for goal placement, clear error messages, and dynamic handling of multiple maps and goals.

6. Conclusion

The lab exercises effectively demonstrated the use of Q learning algorithm for training an agent to navigate from random starting positions to a user-defined goal on given and self-created maps. The lab was divided into two parts: Part 1, which applied Q-learning to a given map, and Part 2, which extended the implementation to allow user-selected maps and goals.

The results demonstrated how parameter tuning influenced the learning process and agent performance. For example, choosing balanced parameters ($\alpha=0.1$, $\gamma=0.9$, $\epsilon=0.1$) ensured steady convergence with rewards improving from negative values to near-optimal outcomes. The use of animations and visualizations like the value function and policy maps helped to visualize the analysis by providing clear insights into the agent's behavior. This lab emphasized the practical implementation of Q-learning by tuning its dependence on parameter choices.